Individual Contribution Report

CMPE 321 — Project 3: Modular DBMS Engine | Boğaziçi University, Spring 2026

Name: Berkay Acer **Student ID:** 2022400066 **Group:** 28

Tasks & Contributions

I was responsible for Phases 0 through 3, covering the full lower-to-middle engine stack.

- **Phase 0 — Skeleton.** Designed the directory layout, wrote all six Result dataclasses in `common/results.py`, and created module stubs with a passing import smoke test.
- **Phase 1 — Lower Stack.** Implemented `DiskSpaceManager` (binary page I/O, free-list header on page 0, I/O counters, `log_write` slot) and `BufferManager` (LRU/MRU via `OrderedDict`, dirty tracking, all five counters). Wrote 14 unit tests.
- **Phase 2 — Upper Stack.** Built the slotted-page encoder/decoder, pickled system catalog, all heap DML operations (`create_type`, `insert_record`, `delete_record`, `search_record`, `range_search`), and the full `QueryProcessor` (dispatcher, `output.txt`, `log.csv`, `stats_output.txt`, `explain`). Added 16 unit tests.
- **Phase 3 — Index Layer.** Implemented the FNV-1a static hash index with overflow chaining and heap-scan fallback for range queries, and the persistent B+-tree with struct-encoded nodes, split propagation, and new-root promotion. Wired both into `FileIndexManager` with lazy caching. Added 14 unit tests; total suite reached 44/44.
- **Video.** Co-recorded the project demonstration video covering architecture and live demo.

Collaboration & Challenges

Enes handled Phase 4 while I owned Phases 0–3. We coordinated through `IMPLEMENTATION_GUIDE.md`, updating it after every design decision. The main challenge was index persistence: Python's built-in `hash()` is salted per run, silently corrupting the hash index on reload. Switching to FNV-1a fixed this. B+-tree split off-by-one errors were isolated with a fanout-3 unit test.

Self-Assessment

This project deepened my understanding of how each abstraction layer isolates complexity. I became comfortable with `struct` pack/unpack and persistence-safe data structures. I would improve by documenting design decisions before coding rather than after.

Use of AI Tools

I used an AI assistant occasionally when I got stuck on a concept and needed a quick explanation. For example, I asked it to clarify how B^+-tree node splitting propagates upward, and to explain the difference between LRU and MRU eviction in the context of sequential workloads. I also used it to look up the correct `struct` format strings for little-endian packing, which I could have found in the documentation but it was faster. I did not use it to write any part of the implementation; everything in the code is something I wrote and understand myself.